

Практика 8. Аппликативные функторы

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Полиморфные типы данных в Haskell (`Maybe`, `[]`, `Either`, `(->)`, ...)
- ▶ Функторы
- + Аппликативные функторы



Содержание

1. Аппликативные функторы
2. Композиция типов
3. Материалы

Содержание

1. Аппликативные функторы

2. Композиция типов

3. Материалы

Аппликативные функторы для вычислений над содержимым коробочек

```
class Applicative f where
  pure  :: a -> f a
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
```

► Применяя функции вида $a \rightarrow f\ b$ получаем значения вида $f\ b$

e.g. `Map.lookup :: k -> Map k v -> Maybe v`

- Хотим продолжать вычисления над значениями в коробочках
- Нужен специальный код для каждого f

todo Реализуйте `liftA3` через `liftA2`

```
⚡ liftA3 :: Applicative f => (a -> b -> c -> d) -> f a -> f b -> f c -> f d
liftA3 f x y z = liftA2 ($) (liftA2 f x y) z
```

Аппликатив Either

```
class Applicative f where
```

```
  pure :: a -> f a
```

```
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
```

todo Реализуйте инстанс аппликатива для **Either**

todo По Id двух игроков, вычислите, у которого преимущество в игре

```
data Player = Player { nick :: String, rank :: Int }
```

```
dbLookup :: Db -> Id -> Either DbError Player
```

todo По двум командам игроков (пара списков Id) вычислите, у которой преимущество



```
advantage :: Db -> [Id] -> [Id] -> Either DbError Bool
```

```
advantage db ids1 ids2 = liftA2 (<) (teamRank ids1) (teamRank ids2) where
```

```
  teamRank = fmap (sumBy rank) . mapA (dbLookup db)
```

```
  sumBy f = sum . map f
```

```
mapA :: Applicative f => (a -> f b) -> [a] -> f [b]
```

```
mapA f = foldr (liftA2 (:)) (pure []) . map f
```

Аппликатив (->)

```
class Applicative f where
```

```
  pure :: a -> f a
```

```
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
```

todo Реализуйте инстанс аппликатива для функциональной стрелки

todo Найдите всех пользователей по Id в надёжной БД

```
dbLookup :: Id -> Db -> Player
```

```
mapA :: Applicative f => (a -> f b) -> [a] -> f [b]
```



```
lookupAll :: [Id] -> Db -> [Player]
```

```
lookupAll = mapA dbLookup
```

Идиома вычислений в аппликативном контексте

- ▶ Хотим использовать функции произвольной аргументности
- ▶ Обобщим “каррированной” версией `liftA2` (на самом деле они эквивалентны)

```
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
```

- ▶ Теперь произвольный `liftAN` выражается однообразно

```
liftA4 :: Applicative f => (a -> b -> c -> d -> r)
      -> (f a -> f b -> f c -> f d -> f r)
```

```
liftA4 f a b c d = pure f <*> a <*> b <*> c <*> d
```

- ▶ `pure f :: f (a -> b -> c -> d -> r)`
 - ▶ `pure f <*> a :: f (b -> c -> d -> r)`
 - ▶ `pure f <*> a <*> b :: f (c -> d -> r)`
 - ▶ `pure f <*> a <*> b <*> c :: f (d -> r)`
 - ▶ `pure f <*> a <*> b <*> c <*> d :: (f r)`
- ▶ Более идеоматично пользоваться `pure f <*> x = f <$> x`
- NB** Аппликативные функторы позволяют замести под ковёр неинтересную рутину и церемонии, оставив нас наедине с бизнес-логикой (в форме идиомы)

Death Context parade (☹)

1. Контекст типизации Γ — хранит типы переменных
2. Контекст типа ($\text{Ord } a$, $\text{Bounded } a$) $\Rightarrow$ — перечень ограничений (constraints) на типовые переменные в этом типе
3. Вычислительный контекст f — типовой конструктор, который объявлен представителем **Applicative**
4. Контекст-окружение $e \rightarrow$ — константная конфигурация, распространяемая реализацией **Applicative** для функциональной стрелки
5. Контекст зиппера — части структуры данных, не находящиеся в фокусе зиппера
6. и т.д.

Так что слово “контекст” мы будем понимать по-разному в зависимости от контекста.

Содержание

1. Аппликативные функторы

2. Композиция типов

3. Материалы

Функции на типах

todo Как написать на типах `id`?



- ▶ Kind: `Id :: * -> *`
- ▶ Def: `type Id a = a`

todo Как написать на типах `flip`?



- ▶ Kind: `Flip :: (* -> * -> *) -> * -> * -> *`
- ▶ Def: `type Flip p a b = p b a`

todo Композиция типов?



- ▶ Kind: `Compose' :: (* -> *) -> (* -> *) -> * -> *`
- ▶ Def: `type Compose' f g a = f (g a)`

todo Композиция типов через `newtype`? В чем разница?



- ▶ Def: `newtype Compose f g a = Compose (f (g a))`
- ▶ Получаем новый тип, эквивалентный (изоморфный) композиции
`Compose f g a ~ f (g a)`

NB Реализации классов типов ищутся для `Compose`, а не для `f`

Композиция типов

```
newtype Compose f g a = Compose { getCompose :: f (g a) }
```

```
todo GHCi> :t Compose [Just 42]
```

```
todo GHCi> :t Compose [Just [42]]
```

```
todo GHCi> :t Compose (Compose [Just [42]])
```

```
⚡ Compose [] Maybe Int
```

```
⚡ Compose [] Maybe [Int]
```

```
⚡ Compose (Compose [] Maybe) [] Int
```

Композиция функторов — функтор

todo Докажите

todo `Compose [Just 1]`

```

⚡ instance (Functor f, Functor g) => Functor (Compose f g) where
    fmap :: (a -> b) -> Compose f g a -> Compose f g b
    --    :: (a -> b) -> f (g a)          -> f (g b)
    fmap f = Compose . fmap (fmap f) . getCompose
GHCi> length <$> Compose [Just [42]]
⚡ Compose [Just 1] :: Compose [] Maybe Int

```

Композиция аппликативов — аппликатив

todo Докажите

```
⚡ instance (Applicative f, Applicative g) => Applicative (Compose f g) where
    pure = Compose . pure . pure
    liftA2 op (Compose x) (Compose y) = liftA2 (liftA2 op) x y
```

- ▶ Теперь пример с игроками можно больше механизировать

```
type Efs = Compose ((->) Db) (Either DbError)
dbLookup :: Id -> Efs Player -- больше не передаём БД вручную
```

Содержание

1. Аппликативные функторы

2. Композиция типов

3. Материалы

Что посмотреть в транспорте

- ▶ Vitaly Bragilevsky - The clear path to Haskell complexities

- ▶ “Applicative Programming with Effects” Conor McBride, 2008¹
- ▶ “Functional Pearl: F for Functor”, Ralf Hunze et al, 2012²

¹Статья, вводящая аппликативы.

²Пьеса(!) о категорном значении функторов.